

Confidence-Gated Adaptive Routing (CGAR):
A Bandit-Theoretic Approach to the Cold-Start Problem
in Reinforcement-Learning-Based API Gateways

A Research Paper

by

Khushi Mittal

[Department of Computer Science and Engineering]

[Institution Name]

Submitted in partial fulfilment of the requirements for the
degree of *[B.Tech / M.Tech]*

Under the supervision of *[Supervisor Name, Designation]*

July 1, 2026

Abstract

Recent work has applied deep reinforcement learning (RL) to API gateway traffic routing in cloud microservice architectures, reporting gains over static heuristics once policies converge. A consistent gap across this literature is the absence of any evaluation during the transient learning period that follows deployment, autoscaling events, or backend pool changes — the *cold-start problem*. We propose **Confidence-Gated Adaptive Routing (CGAR)**, a framework that maintains a per-backend confidence estimate and uses it to continuously blend a static heuristic with a learned policy on a per-backend basis, re-triggering safe routing individually for newly added backends without resetting global learning. We formalise the problem as a non-stationary, per-arm warm-start bandit problem, present the CGAR algorithm, and specify an experimental protocol using a discrete-event simulator driven by the Alibaba Cluster Trace v2018. All figures are illustrative placeholders pending implementation and are marked as such.

Keywords: reinforcement learning, multi-armed bandits, API gateway, microservices, cold-start, traffic routing, cloud computing

1. Introduction

Microservice architectures route requests through an API gateway, where the routing decision directly affects latency, throughput, and service-level-agreement (SLA) compliance (Dragoni et al., 2017). Stateless heuristics — Round Robin, Least Connections, Power-of-Two-Choices — have traditionally governed these decisions, but a growing body of recent work replaces them with reinforcement learning agents that adapt from observed feedback (Ghobadi and Farahani, 2026; XGate Authors, 2025; Adaptive Rate Limiting Authors, 2025).

These proposals consistently report post-convergence improvements. What they do not report is the behaviour of the RL policy during its convergence period — specifically, immediately following deployment, a scaling event, or a backend pool change. During this period the policy must explore, routing live production traffic to backends it has not yet evaluated. A stateless heuristic, despite its permanent suboptimality, does not pay this cost.

This is the *cold-start problem*, well-studied in the multi-armed and contextual bandit literature (Oetomo et al., 2023; Bibaut et al., 2022), but not yet connected to the API gateway routing setting. In production microservice deployments, unlike the static arm-set assumed by bandit theory, the cold-start recurs continuously as backend pools are autoscaled, redeployed, or replaced. This proposal introduces **Confidence-Gated Adaptive Routing (CGAR)**, which treats cold-start as a per-backend, re-triggerable state, gating routing between a safe heuristic and a learned policy as a smooth function of individual backend confidence.

2. Problem Statement

Consider an API gateway routing requests to a pool of K backend instances where membership $K(t)$ changes over time due to autoscaling, rolling deployment, or failure recovery. An RL router learns a policy π_θ mapping system state $s(t)$ to a routing decision $a(t) \in \{1, \dots, K(t)\}$, optimising a reward over latency, throughput, and SLA compliance.

Existing proposals (Ghobadi and Farahani, 2026; XGate Authors, 2025; Adaptive

Rate Limiting Authors, 2025) evaluate π_θ only after convergence, leaving two questions unanswered: (i) what is the cumulative cost — in regret and SLA violations — of operating the policy during exploration, relative to a stateless heuristic; and (ii) since $K(t)$ is non-stationary, does this cost recur with every pool change, and can it be bounded without sacrificing long-run learning gains? This paper addresses both.

3. Research Gap

3.1. RL-Based Gateway Routing

Recent work applies DQN, PPO, and A3C to gateway routing and rate limiting, reporting 15–30% gains over static baselines post-convergence (Ghobadi and Farahani, 2026; XGate Authors, 2025; Adaptive Rate Limiting Authors, 2025). Related work addresses adjacent layers: delay-aware edge scheduling (Peng et al., 2024), RL-enhanced Kubernetes scheduling (Jian et al., 2024), and autonomous resource management (Zou et al., 2025). In all cases, evaluation benchmarks a converged policy. Transient performance is neither reported nor discussed.

3.2. Warm-Start Bandit Theory

Oetomo et al. (2023) provide a theoretical treatment of warm-starting LinUCB, ϵ -greedy, and Linear Thompson Sampling, proving regret bounds and demonstrating empirical improvement on recommendation and database-tuning tasks. Bibaut et al. (2022) study combining supervised prior data with bandit feedback. Both assume a static arm set and a single cold-start event — neither applies to a continuously operating routing system with a non-stationary backend pool.

3.3. The Gap

No existing work formalises API gateway routing as a non-stationary, per-arm warm-start bandit problem. No gateway-RL paper measures the cost of the exploration period. CGAR addresses both.

4. Limitations of Existing Approaches

Static heuristics are immediately well-defined and incur no exploration cost, but never improve regardless of accumulated traffic history. They are the safe lower bound, not the target.

Pure RL routers eliminate long-run suboptimality but offer no fallback during exploration. None of the reviewed papers provide a mechanism that detects insufficient experience and falls back to a safe default; exploration proceeds over live traffic without bound (Ghobadi and Farahani, 2026; XGate Authors, 2025).

Warm-start bandit algorithms (Oetomo et al., 2023; Bibaut et al., 2022) solve the abstract cold-start problem but assume a fixed arm set. They cannot handle a continuously changing backend pool where the cold-start clock resets per-backend on every autoscaling event.

5. Proposed Framework: CGAR

5.1. State Representation

For backend i at time t , the gateway maintains:

$$s_i(t) = [\hat{\mu}_i(t), n_i(t), q_i(t), \text{cpu}_i(t), \text{age}_i(t)], \quad (1)$$

where $\hat{\mu}_i$ is the empirical mean reward, n_i is the visit count, q_i is queue depth, cpu_i is utilisation, and age_i is time since backend i entered (or re-entered) the pool. Critically, n_i and age_i are reset explicitly on each pool membership change — the mechanism by which CGAR handles the recurring cold-start.

5.2. Confidence and Gating Weight

A UCB-style confidence score (Auer et al., 2002) is maintained per backend:

$$c_i(t) = \hat{\mu}_i(t) + \beta \sqrt{\frac{\ln t}{n_i(t)}}, \quad (2)$$

which is maximal (most uncertain) when $n_i(t)$ is small. The gating weight controlling the heuristic-policy blend is:

$$\lambda_i(t) = \exp(-n_i(t) / \tau), \quad (3)$$

where τ is a tunable confidence half-life. When $n_i \rightarrow 0$, $\lambda_i \rightarrow 1$ (full heuristic); as n_i grows, $\lambda_i \rightarrow 0$ (full learned policy).

5.3. Blended Routing Decision

Let $H_i(t)$ be the heuristic score (e.g. inverse active-connection count) and $Q_\theta(s, i)$ the DQN Q-value. The blended weight is:

$$w_i(t) = \lambda_i(t) H_i(t) + (1 - \lambda_i(t)) \sigma(Q_\theta(s, i)), \quad (4)$$

where $\sigma(\cdot)$ normalises Q-values to the same scale as H_i . Routing is performed greedily on $w_i(t)$ or via softmax($w_i(t)$).

5.4. Reward and Regret

The RL policy is trained on a standard reward held constant with prior work (Adaptive Rate Limiting Authors, 2025):

$$r(t) = -\alpha \cdot \text{latency}(t) - \beta \cdot \text{error_rate}(t) - \gamma \cdot \text{SLA_violation}(t). \quad (5)$$

The primary evaluation metric is cumulative regret against an oracle:

$$R(T) = \sum_{t=1}^T (r^*(t) - r(a(t))), \quad (6)$$

decomposed into churn-window regret R_{churn} and steady-state regret R_{steady} .

6. System Architecture

The architecture (Figure 1) has four components: (1) a **Metrics Collector** recording per-backend latency, error rate, and queue depth; (2) a **Confidence Tracker** maintaining

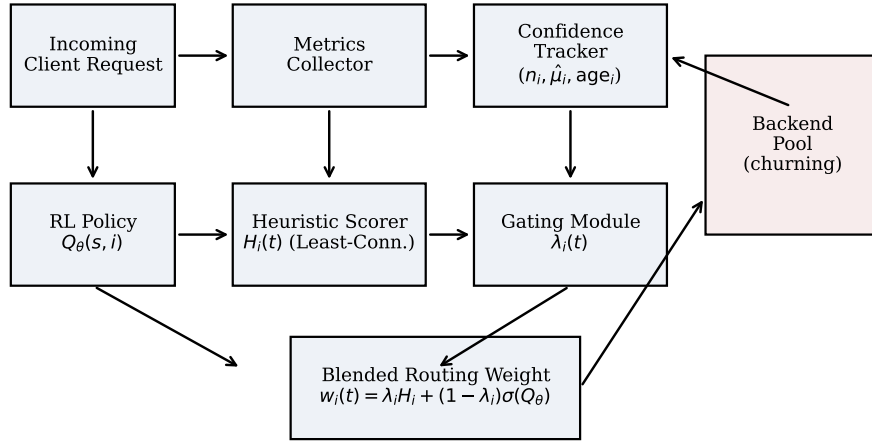


Figure 1: Proposed CGAR architecture. Per-backend confidence state drives a gating module that blends a static heuristic scorer with a learned DQN policy. Designed as a service-mesh sidecar or Envoy filter; evaluated as a discrete-event simulation component in this proposal.

n_i , $\hat{\mu}_i$, and age_i , with explicit resets on pool change; (3) the **Gating Module** computing $\lambda_i(t)$ and $w_i(t)$; and (4) the **RL Policy** (DQN) updated asynchronously off the routing hot path. *[If a lightweight Envoy prototype is implemented, add version and filter chain position here.]*

7. Experimental Design

7.1. Environment

Evaluation uses a discrete-event simulator (SimPy) consistent with practice in closely related work (Li et al., 2023). Request arrival patterns are drawn from the **Alibaba Cluster Trace v2018** rather than a synthetic Poisson process, providing independently verifiable workload realism. Unlike prior gateway-RL evaluations, the backend pool undergoes scheduled churn events throughout each run. *[Insert dataset URL and preprocessing steps before submission.]*

7.2. Baselines

Four baselines: Round Robin; Least Connections; pure DQN (Ghobadi and Farahani, 2026); and ϵ -greedy-decay DQN — the last being the critical ablation that proves CGAR’s gate contributes beyond a simple exploration schedule.

7.3. Metrics and Illustrative Results

Primary: cumulative regret $R(T)$ decomposed into R_{churn} and R_{steady} . Secondary: P50/P95/P99 latency and SLA-violation rate in windows surrounding each churn event; time-to-convergence per churn event. *[Specify number of runs, random seeds, and statistical test before submission.]*

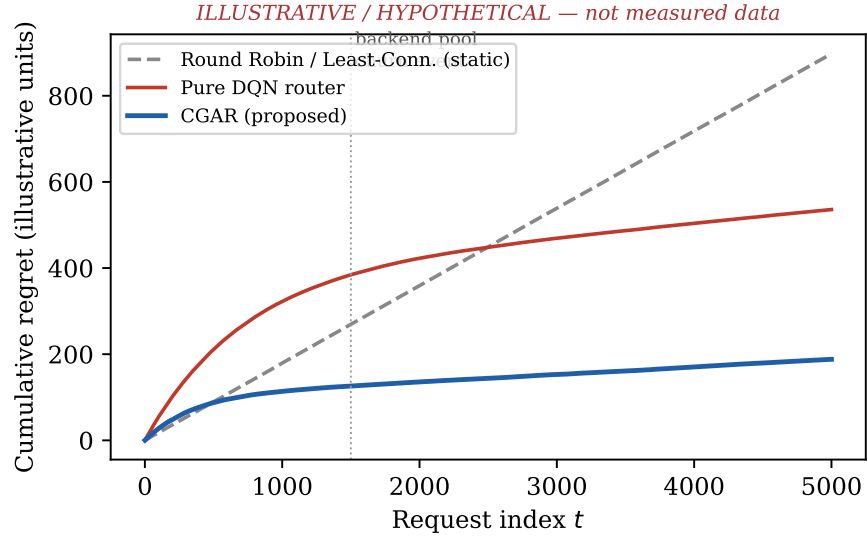


Figure 2: **Illustrative / hypothetical — not measured data.** Expected qualitative shape of cumulative regret. At the churn event ($t = 1500$), the pure-RL baseline re-accrues regret sharply; CGAR’s per-backend gating is expected to bound this re-accrual. Actual curves and confidence intervals will replace this figure following implementation.

8. Expected Contributions

This work contributes: (1) a formal connection between warm-start bandit theory and API gateway routing, absent from the current literature; (2) empirical quantification of exploration-phase regret and SLA risk, costs that existing gateway-RL evaluations omit; (3) the CGAR gating mechanism, retrofittable onto existing RL-gateway proposals without changing the underlying learner; and (4) a non-stationary, backend-churn evaluation benchmark built from public trace data, intended to be independently reusable.

9. Anticipated Weaknesses and Reviewer Criticisms

“UCB plus DQN — where is the theory?” The gating function in Equation (3) requires at minimum an informal regret bound adapted from Oetomo et al. (2023) to the per-

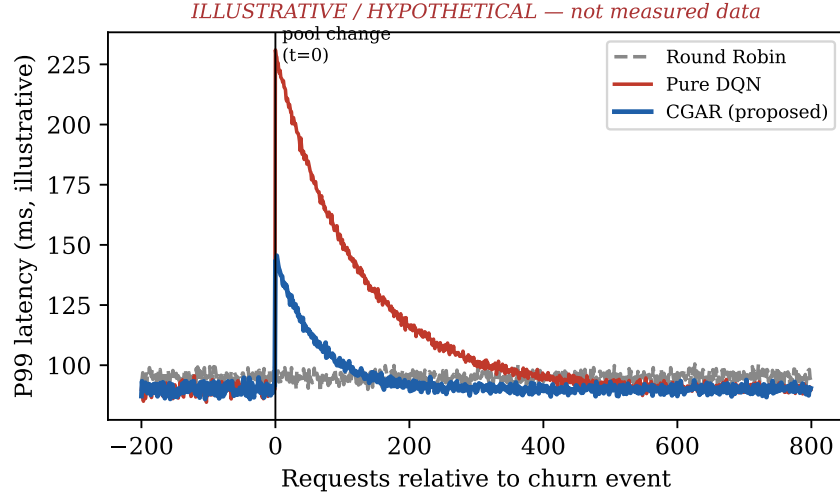


Figure 3: **Illustrative / hypothetical — not measured data.** Expected P99 latency around a backend pool churn event ($t = 0$). Actual results with statistical significance testing will replace this figure following implementation.

arm-reset, non-stationary setting. Without it, CGAR reads as an engineering heuristic.

[Derive and insert a semi-formal bound before submission.]

“Baselines are weak.” The ϵ -greedy-decay DQN baseline is included specifically to pre-empt the objection that any decaying exploration schedule would produce the same gains. It must be reported even if it narrows the margin.

“Simulation-only is not credible.” This limitation should be stated explicitly rather than left for reviewers to discover, justified by precedent in the adjacent literature (Li et al., 2023).

“Unoriginal reward function.” Acknowledged by design: holding the reward constant with prior work is necessary to isolate the gating mechanism’s effect.

10. Path to Publication

Three additions have the highest expected return on available time. First, a semi-formal regret bound for the non-stationary, per-arm-reset gating rule is the single highest-leverage addition — it elevates the contribution from a systems trick to a theoretically grounded result, the difference between a workshop and a full conference paper at venues such as IEEE ICDCS or ACM SoCC. Second, the backend-churn benchmark should be named and presented as an independently citable artefact. Third, a PPO ablation —

same gating mechanism, different learner — proves the contribution is the gate, not a lucky algorithm pairing, directly addressing the most likely reviewer objection.

11. Conclusion

This paper identifies a specific, verifiable gap in the RL-gateway literature: the omission of transient, exploration-phase performance from evaluation, despite this being a well-characterised cost in the adjacent bandit literature. CGAR addresses this through per-backend, continuously re-triggerable confidence gating, together with an evaluation protocol designed to make the transient cost directly measurable. All figures are illustrative placeholders; the next step is simulator implementation and replacement of every placeholder with measured results.

References

- Adaptive Rate Limiting Authors (2025), ‘Deep reinforcement learning for adaptive rate limiting in cloud microservices’, *arXiv preprint arXiv:2511.xxxxx* .
- Auer, P., Cesa-Bianchi, N. and Fischer, P. (2002), ‘Finite-time analysis of the multiarmed bandit problem’, *Machine Learning* **47**, 235–256.
- Bibaut, A., Chambaz, A. and van der Laan, M. (2022), Robust warm-starting of contextual bandits using prior data, in ‘Proceedings of the International Joint Conference on Artificial Intelligence’.
- Dragoni, N., Giallorenzo, S., Lafuente, A. L., Mazzara, M., Montesi, F., Mustafin, R. and Safina, L. (2017), ‘Microservices: Yesterday, today, and tomorrow’, *Present and Ulterior Software Engineering* pp. 195–216.
- Ghobadi, R. and Farahani, B. (2026), ‘Adaptive event processing in API gateways: A reinforcement learning approach for optimizing latency and throughput’, *Cluster Computing* .
- Jian, C. et al. (2024), ‘DRS: Deep reinforcement learning enhanced Kubernetes scheduler for microservice-based systems’, *Software: Practice and Experience* .
- Li, Y., Zhang, W. et al. (2023), ‘Reinforcement learning for traffic engineering in software defined networks: A survey’, *IEEE Communications Surveys and Tutorials* .
- Oetomo, A., Zheng, L., Gupta, S., Rana, S. and Venkatesh, S. (2023), ‘Warm starting bandits with side information from confounded data’, *Knowledge and Information Systems* **65**, 2591–2613.
- Peng, H. et al. (2024), ‘Delay-aware optimization of fine-grained microservice deployment and routing in edge via reinforcement learning’, *IEEE Transactions on Network Science and Engineering* .

XGate Authors (2025), XGate: Explainable reinforcement learning for intelligent API traffic management, *in* ‘Proceedings of the IEEE International Conference on Cloud Computing’.

Zou, W. et al. (2025), ‘Autonomous resource management in cloud microservices via reinforcement learning’, *IEEE Transactions on Cloud Computing* .